

Lab Manual

OilRig / BONDUPDATER

Elastic/Kibana Incident Investigation

Scenario: You are a SOC analyst reviewing Elastic telemetry from a Windows endpoint. The host generated several alerts after a Microsoft Office document was opened. You are not initially told the malware family. Your job is to reconstruct the intrusion from evidence, identify persistence, find the C2 mechanism, and prepare a concise incident report.

Recommended index names (Optional): Use the following names so the lab queries are consistent. If you already used different names, adjust the `_index` filter in each query.

Upload file	Index name
ep7data.ndjson	logs-ommep7events
ep7alerts.ndjson	logs-ommep7alerts

Upload steps

- Download ep7data.ndjson.gz and decompress it to ep7data.ndjson.
- In Kibana, go to Home -> Upload a file.
- Upload ep7data.ndjson and click Import.
- Open the Advanced tab.
- Set the index name to logs-ommep7events.
- Add this index setting: "mapping.total_fields.limit":"6000".
- Replace the generated mapping with the contents of mapping.json.
- Click Import.

Repeat the same process for ep7alerts.ndjson using index name logs-ommep7alerts and the same mapping.json.

Create the data view

Data view name: Data View – ep7

Index pattern: logs-ommep7*

Timestamp field: @timestamp

Time picker: Set Kibana time to Last 10 years first. After confirming data exists, narrow to the known activity window:
2023-04-01 22:00:00 UTC -> 2023-04-01 22:15:00 UTC

1) Questions:

Which alert gives the first reliable investigation lead?

Which application appears to be the parent process?

Which child process is launched?

What script path is used?

KQL: `_index : "logs-ommep7alerts"`

Add columns

@timestamp
rule.name
event.severity
process.parent.name
process.parent.command_line
process.name
process.command_line
file.path
host.name
user.name

Expected result: Students should identify alerts such as Suspicious Microsoft Office Child Process and Script File Written by Microsoft Office Process. The strongest lead is Microsoft Word launching wscript.exe and writing script content under C:\ProgramData\WindowsAppPool.

2) Identify the malicious document and suspicious child process

Question: What document started the suspicious process chain?

KQL

`_index : "logs-ommep7alerts" and process.parent.name : "WINWORD.EXE"`

Expected evidence

Parent process: WINWORD.EXE

Parent command line contains:

C:\Users\Administrator\Desktop\9828375091\7cbad6b3f505a199d6766a86b41ed23786bbb99dab9cae6c18936afdc2512f00.doc

Child process: wscript.exe

Child command line: wscript C:\ProgramData\WindowsAppPool\AppData.vbs

3) Pivot into raw events for the Word-to-script chain

Question: Can we prove the alert with raw endpoint events?

KQL: `_index : "logs-ommep7events" and
(process.parent.name : "WINWORD.EXE" or process.name : "WINWORD.EXE" or process.name :
"wscript.exe")`

Alternate query: `_index : "logs-ommep7events" and process.command_line : *AppPool.vbs*`

Add columns

@timestamp
event.category
event.action
process.parent.name
process.parent.command_line
process.name
process.command_line
process.executable
file.path

Expected result: Students should see wscript.exe executing C:\ProgramData\WindowsAppPool\AppData.vbs. They may also see surrounding file events for script creation in the same directory.

4) Find dropped script files

Question:

What files were dropped to disk?

Why is C:\ProgramData a suspicious staging location?

KQL `_index : "logs-ommep7events" and file.path : *WindowsAppPool*`

Add file.name field.

5) Investigate scheduled-task persistence

Question:

Did the malware create a persistence mechanism?

Which part of the command shows persistence frequency?

KQL: `_index : "logs-ommep7events" and process.command_line : *schtasks* and process.command_line :
WindowsAppPool`

6) Follow PowerShell execution

Question: How does the VBScript launch the main payload?

KQL: _index : "logs-ommep7events" and process.name : "powershell.exe" and process.command_line : *AppPool.ps1*

Expected evidence

PowerShell.exe -exec bypass -file C:\ProgramData\WindowsAppPool\AppData.ps1

7) Investigate BONDUPDATER working files and folders

Question

What local artifacts does the payload create for state, staging, and data transfer?

KQL: _index : "logs-ommep7events" and file.path : *WindowsAppPool* and (file.path : *lock* or file.path : *quid* or file.path : *receivebox* or file.path : *sendbox* or file.path : *done* or file.path : *files*)

Expected artifacts to look for

C:\ProgramData\WindowsAppPool\lock
C:\ProgramData\WindowsAppPool\quid
C:\ProgramData\WindowsAppPool\files
C:\ProgramData\WindowsAppPool\<unique_id>\receivebox
C:\ProgramData\WindowsAppPool\<unique_id>\sendbox
C:\ProgramData\WindowsAppPool\<unique_id>\done

8) Identify DNS-based command and control

Question: Did the payload communicate over DNS?

KQL

_index : "logs-ommep7events" and network.protocol : "dns" and process.name : "powershell.exe"

Add columns

@timestamp
process.name
process.command_line
network.protocol
dns.question.name
dns.question.type
dns.answers.data
destination.ip
destination.port

Expected result

Students should identify DNS queries involving withyourface.com or subdomains under withyourface.com. These subdomains may look long, randomized, or structured because BONDUPDATER embeds communication data into DNS queries.

9) **Evidence table:** Complete the following evidence table:

Evidence type	Expected evidence
Initial file	7cbad6b3f505a199d6766a86b41ed23786bbb99dab9cae6c18936afdc2512f00.doc
Parent process	WINWORD.EXE
Script host	wscript.exe C:\ProgramData\WindowsAppPool\AppData.vbs
Dropped scripts	C:\ProgramData\WindowsAppPool\AppData.vbs and AppPool.ps1
Persistence command	schtasks /create /F /sc minute /mo 1 /tn "\WindowsAppPool\AppData" /tr "wscript /b ...AppPool.vbs"
PowerShell payload	PowerShell.exe -exec bypass -file C:\ProgramData\WindowsAppPool\AppData.ps1
Working artifacts	lock, quid, files, receivebox, sendbox, done
C2 domain	withyourface[.]com